axi2apb_bridge: AXI4-Lite to APB3 Bridge

Design document - verified with Verilator, synthesised and timed with Yosys and OpenSTA (Sky130 HD, pre-layout)

A. Project overview

A.1 Purpose

`axi2apb` is a synthesisable bridge that lets an AXI4-Lite master reach an APB3 peripheral. AXI4-Lite is the register-access flavour of AXI (five channels, single beats, no bursts), and APB3 is the low-power peripheral bus (a two-phase transfer with wait states and an error flag). The bridge converts one into the other: it accepts an AXI read or write, performs one APB transfer, and returns the AXI response.

A.2 Problem statement and motivation

The two protocols have different shapes. AXI has independent address, data and response channels, each with its own valid/ready handshake, while APB has a single shared bus with a setup phase and an access phase that lasts until the slave raises PREADY. So a bridge has to (1) serialise the AXI channels into APB phases, (2) hold the AXI response stable until the master takes it, (3) honour APB wait states, and (4) turn the APB error signal into the right AXI response code. What I wanted from this project was to make that conversion provably correct, because the original version of this bridge returned the wrong AXI response code for an APB error and nothing in its testbench could notice.

A.3 Scope and requirements

ID	Requirement
R1	AXI4-Lite slave interface (AR, R, AW, W, B), `DATA_WIDTH` and `ADDR_WIDTH` parameters (default 32)
R2	APB3 master interface (PSEL, PENABLE, PWRITE, PADDR, PWDATA, PREADY, PRDATA, PSLVERR)
R3	Correct APB phases: a setup phase, then an access phase that lasts until PREADY; address, direction and write data stable throughout
R4	AXI responses held until taken (BVALID/RVALID with stable BRESP/RDATA/RRESP until BREADY/RREADY)
R5	APB error (PSLVERR) returned as SLVERR (2'b10); success as OKAY (2'b00); no other response code
R6	One transaction at a time; a read request has priority over a simultaneous write request
R7	Verified by a self-checking testbench, bound protocol assertions, coverage and an injected-bug campaign; synthesised and timed on Sky130 HD

Out of scope: write strobes (the bridge has no WSTRB input, so every write is a full-word write), APB4 (PSTRB/PPROT), AXI PROT, more than one outstanding transaction, and clock-domain crossing.

A.4 Target application and concepts

The bridge connects low-speed peripherals (timers, UARTs, GPIO, register files) to an AXI interconnect. Building and checking it covers the AMBA AXI4-Lite and APB3 handshake rules, protocol conversion with an FSM, wait-state handling, response mapping, protocol assertions, coverage, mutation analysis, synthesis and timing.

A.5 Key features and measurable objectives

- An 8-state Moore FSM. Without wait states the write path takes 5 cycles and the read path 4 cycles from IDLE back to IDLE (observed in simulation).
- 23 bound assertions and 17 named cover points, a 15-bug injection campaign, and the as-found RTL shown failing the new testbench (section G).
- Objective: every check passes on 8 seeds x 3000 random transactions plus the directed phases, every cover point and line is hit, and each synthesis mapping has a measured T_min and area.

B. Architecture and block diagram

B.1 Overview

The bridge is one module (`src/axi2apb.v`) with a control FSM, some output decode and a few registers. The FSM state decides which AXI ready/valid signals and which APB signals are asserted, and the registers capture the address, drive PADDR/PWDATA, and hold the read data and the AXI response. There's no FIFO and no skid buffer: the handshake with the AXI master is just the FSM waiting in the right state.

B.2 Blocks

Block	Responsibility
Control FSM	`present_state` / `next_state`; arbitration (read first); wait for WVALID, PREADY, BREADY, RREADY
Output decode	ARREADY, AWREADY, WREADY, RVALID, BVALID, PSEL, PENABLE, PWRITE as functions of `present_state` (Moore outputs)
`sampled_addr`	latches ARADDR or AWADDR in IDLE
PADDR / PWDATA registers	drive the APB address and write data; loaded when the write data is accepted, or when a read starts
Response registers	RDATA <= PRDATA (if no error); BRESP / RRESP <= PSLVERR ? SLVERR : OKAY, at the PREADY cycle
`sampled_wdata`	a register that captures WDATA but whose output is not used (lint reports it; synthesis removes it)

B.3 Interfaces

Group	Signals	Notes
clock/reset	`clk`, `rst_n`	synchronous active-low reset
AXI read	`S_AXI_ARADDR`, `ARVALID`, `ARREADY`, `RDATA`, `RRESP`, `RVALID`, `RREADY`	single beat
AXI write	`S_AXI_AWADDR`, `AWVALID`, `AWREADY`, `WDATA`, `WVALID`, `WREADY`, `BRESP`, `BVALID`, `BREADY`	AWREADY = WREADY (asserted together)
APB	`M_APB_PADDR`, `PSEL`, `PENABLE`, `PWRITE`, `PWDATA`, `PREADY`, `PRDATA`, `PSLVERR`	APB3

Parameters: `DATA_WIDTH` (32), `ADDR_WIDTH` (32).

B.4 Data and control paths

Data path: `ARADDR`/`AWADDR` -> `sampled_addr` -> `PADDR`; `WDATA` -> `PWDATA`; `PRDATA` -> `RDATA`. **Control path**: the AXI valids, `WVALID`, `PREADY`, `BREADY` and `RREADY` drive `next_state`, and the state drives the AXI readies and valids and the APB select, enable and write signals. **Status path**: `PSLVERR` and `PREADY` come back from the slave and are registered into the response registers in the PREADY cycle.

C. Draw.io architecture diagram

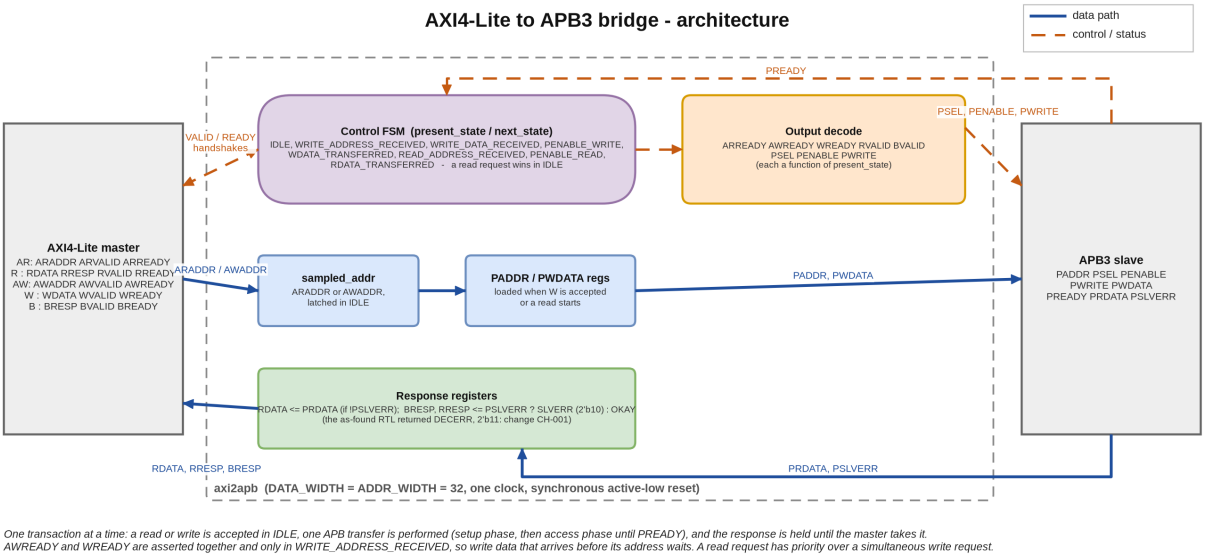


Figure C.1 - AXI4-Lite to APB3 bridge: architecture (data flow solid blue, control flow dashed orange)

The diagram files are in `docs/portfolio/diagrams/`: `axi2apb_architecture.drawio` (editable), `.png` and `.svg`, plus `axi2apb_flowchart.*`. The images are drawn by `dgm.py` (next to the diagram script) from the same specification that writes the `.drawio` file, so they aren't exports from the draw.io application.

D. Design flowchart

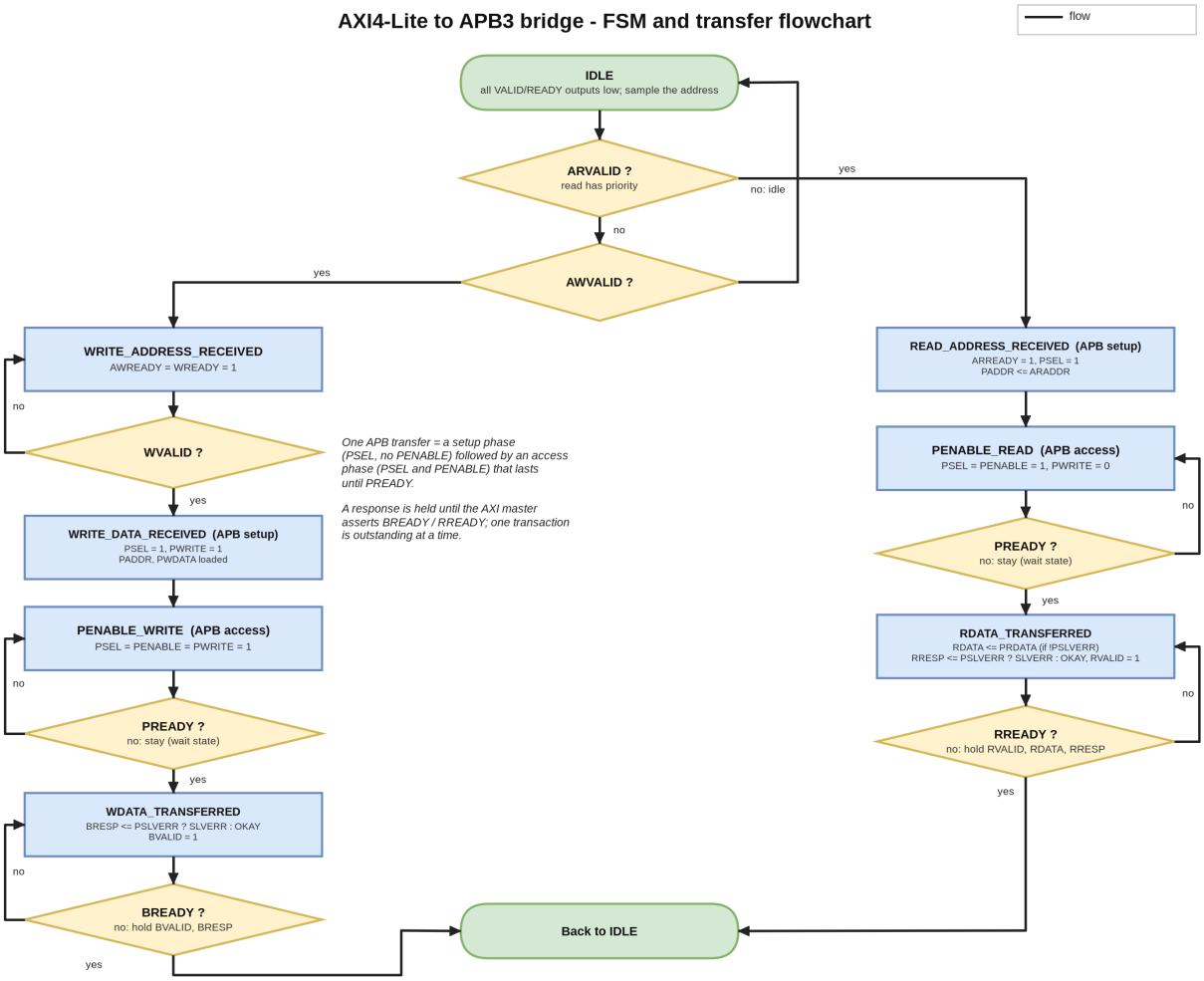


Figure D.1 - FSM and transfer flowchart: IDLE, write path (left), read path (right), wait loops and responses

In IDLE, a read request (ARVALID) wins over a write request (AWVALID), and with neither the bridge stays in IDLE. A write goes through WRITE_ADDRESS_RECEIVED (AWREADY = WREADY = 1, waiting for WVALID), WRITE_DATA_RECEIVED (the APB setup phase), PENABLE_WRITE (the APB access phase, waiting for PREADY) and WDATA_TRANSFERRED (BVALID held until BREADY). A read goes through READ_ADDRESS_RECEIVED (ARREADY = 1 and the APB setup phase at the same time), PENABLE_READ (waiting for PREADY) and RDATA_TRANSFERRED (RVALID held until RREADY). Every "no" branch means staying in the same state.

E. Detailed design explanation

E.1 Step-by-step operation

Write. (1) In IDLE the bridge sees AWVALID and latches the address. (2) In WRITE_ADDRESS_RECEIVED it asserts AWREADY and WREADY and waits for WVALID; PADDR and PWDATA are loaded when WVALID arrives. (3) In WRITE_DATA_RECEIVED it asserts PSEL and PWRITE, which is the APB setup phase. (4) In PENABLE_WRITE it adds PENABLE, and this access phase lasts until PREADY. (5) In the PREADY cycle BRESP is registered from PSLVERR. (6) In WDATA_TRANSFERRED BVALID is asserted and held until BREADY, and then the FSM goes back to IDLE.

Read. (1) In IDLE the bridge sees ARVALID and loads PADDR with ARADDR. (2) In READ_ADDRESS_RECEIVED it asserts ARREADY and PSEL together (the APB setup phase). (3) In PENABLE_READ it adds PENABLE until PREADY; RDATA is captured from PRDATA when there's no error, and RRESP is registered from PSLVERR. (4) In RDATA_TRANSFERRED RVALID is asserted and held until RREADY.

These are the state sequences from the FSM definition, with no APB wait state, W valid together with AW, and a master that's ready at once: a write goes IDLE -> WRITE_ADDRESS_RECEIVED -> WRITE_DATA_RECEIVED -> PENABLE_WRITE -> WDATA_TRANSFERRED -> IDLE (5 cycles), and a read goes IDLE -> READ_ADDRESS_RECEIVED -> PENABLE_READ -> RDATA_TRANSFERRED -> IDLE (4 cycles). Each APB wait state adds one cycle in PENABLE_WRITE or PENABLE_READ, and a slow master adds cycles in the response states.

E.2 Response mapping and the fix made in this portfolio copy (CH-001)

The as-found RTL set BRESP and RRESP to `2'b11` when PSLVERR was high. In AXI, `2'b11` is DECERR, a decode error meaning there's no slave at the address. An APB slave that signals an error is reporting a slave error, which is SLVERR, `2'b10`, and an APB bridge has no decode error to report. I changed those two lines and nothing else in the RTL. Section G.7 shows the effect by running the as-found RTL against the new testbench.

E.3 Reset, latency, throughput and errors

- **Reset**: synchronous. The FSM goes back to IDLE and the address, data and response registers clear.
- **Latency**: 5 cycles for a write and 4 for a read from acceptance back to IDLE, plus any APB wait states and master back-pressure.
- **Throughput**: one transaction at a time, so at best one transfer every 4 or 5 cycles. The bridge is meant for register access, not bulk transfer.
- **Back-pressure**: BVALID and RVALID (with BRESP, RRESP and RDATA) are held until the master takes them, and the APB access phase is held until PREADY.
- **Errors**: an APB error becomes SLVERR on the matching response, and a read error doesn't update RDATA.

E.4 Design decisions and trade-offs

Decision	Alternative	Reason / cost
Moore FSM, one state per phase	Mealy outputs	glitch-free registered-state outputs, easy to assert on; costs a cycle per phase
Read priority in IDLE	round-robin	simple and documented; a continuous read stream could starve writes (not an issue for register access)
AWREADY = WREADY together	independent channels	a W that arrives before AW simply waits; avoids buffering write data
Single outstanding transaction	pipelined / outstanding	smallest bridge; sufficient for peripheral registers
Binary state encoding	one-hot	unchanged from the original; encoding was not part of the portfolio changes

---

F. Interface and timing details

F.1 APB phases

A transfer is a **setup phase** (PSEL = 1, PENABLE = 0) of exactly one cycle, followed by an **access phase** (PSEL = PENABLE = 1) that lasts until PREADY = 1. During the access phase PADDR, PWRITE and, for a write, PWDATA mustn't change. These rules, and the rule that PENABLE drops after the transfer completes, are checked by assertions (section G.4).

F.2 AXI response rules

Once BVALID is asserted it stays asserted, with BRESP stable, until BREADY; the same goes for RVALID with RDATA and RRESP until RREADY. OKAY and SLVERR are the only legal responses.

F.3 Clocking and reset

One clock domain, rising-edge flops and a synchronous active-low reset. There's no clock-domain crossing.

F.4 Timing constraints used in synthesis

The constraints assume inputs arrive 0.35 ns after the clock edge and outputs are needed 0.15 ns before the next edge, with inputs driven by a `buf_2` and an output load of 0.02 pF (`constraints/axi2apb.sdc`). Typical corner, ideal clock, no wires.

F.5 Supported conditions and limitations

One outstanding transaction. Masters have to hold valid and the payload until ready, and the APB slave has to hold PRDATA and PSLVERR valid in the cycle PREADY is high (APB3). A slave that asserts PREADY in the setup phase isn't covered.

G. Verification and testing

G.1 Objectives and test plan

Show that every read returns the last value written, that every response code is right, that the APB and AXI protocol rules are never broken, that the stimulus reaches the interesting corners, and that the checks can fail.

G.2 Testbench architecture

`tb_axi2apb` has five free-running channel processes (AW, W, B, AR, R), each fed from a queue with a delay per item. Around them sit an APB slave model (`apb_slave_model`: 64 words, programmable wait states, and an error region where `paddr[11:8] == 4'hF`) and a scoreboard with a 64-word reference memory and in-order expectation queues. The queue-fed processes replaced a first version built from `fork`/`join` of tasks, which crashed Verilator 5.020.

G.3 Test phases

(1) Directed writes and reads; (2) APB wait states of 1 to 3 cycles; (3) error responses for reads and writes, each followed straight away by a good access; (4) back-pressure on B and R; (5) W before AW, and AW before W; (6) a read and a write requested together, then (6b) pipelined bursts in which the next request arrives while the previous response is still pending; and (7) a constrained-random mix of writes, reads, bursts and simultaneous read/write pairs, with random delays and wait states and an error region hit 15 % of the time.

G.4 Assertions

`verif/axi2apb_checker.sv` holds 23 bound assertions. For APB3: `a_penable_needs_psel`, `a_setup_then_access`, `a_access_until_ready`, `a_penable_drops`, `a_first_cycle_is_setup`, `a_addr_stable`, `a_dir_stable`, `a_wdata_stable` and `a_setup_to_access_same`. For the AXI channels: `a_bvalid_holds`, `a_rvalid_holds`, `a_bresp_legal`, `a_rresp_legal`, `a_aw_w_ready_together` and `a_no_resp_both`. For the response mapping: `a_bresp_maps_slverr`, `a_rresp_maps_slverr` and `a_rdata_is_prdata`. For sequencing: `a_one_outstanding`, `a_bvalid_only_after_write` and `a_rvalid_only_after_read`. And for arbitration: `a_read_priority` and `a_write_when_no_read`. The 17 named cover points cover writes and reads that end OK and in error, wait states, back-pressure, W-before-AW, AW-before-W, simultaneous AR/AW, and back-to-back writes.

G.5 Results: regression

```
8/8 seeds pass (3000 random items each, plus the directed phases)
seed 1: TB PASS checks=4983 writes=2711 reads=2272 error_responses=765
seed 2: TB PASS checks=5034 writes=2701 reads=2333 error_responses=713
seed 3: TB PASS checks=4944 writes=2745 reads=2199 error_responses=710
seed 4: TB PASS checks=4970 writes=2703 reads=2267 error_responses=747
seed 5: TB PASS checks=4948 writes=2677 reads=2271 error_responses=732
seed 6: TB PASS checks=4997 writes=2730 reads=2267 error_responses=710
seed 7: TB PASS checks=5044 writes=2742 reads=2302 error_responses=765
seed 8: TB PASS checks=4988 writes=2766 reads=2222 error_responses=732
```

G.6 Coverage

```
Merged coverage of 8 seeds x 2000 random items + the directed phases (results/coverage_summary.txt):
  named cover points : 17 total, 17 hit, 0 zero
  axi2apb.v line blocks: 34 total, 34 covered, 0 uncovered
```

G.7 The as-found RTL against the same testbench

```
As-found RTL (src/original/axi2apb_original.v) against the same testbench and checker, seed 1, 3000 random items:
  result: TB FAIL checks=4983 errors=765
  testbench errors: 20
  assertions fired (name: times): a_bresp_legal: 404, a_bresp_maps_slverr: 403, a_rresp_legal: 364, a_rresp_maps_slverr: 362
  first testbench errors:
    TB ERROR @2765000: write to error address 00000f20: BRESP=11, expected 10
    TB ERROR @2875000: read of error address 00000f24: RRESP=11, expected 10
    TB ERROR @2985000: write to error address 00000f28: BRESP=11, expected 10
    TB ERROR @3045000: read of error address 00000f2c: RRESP=11, expected 10
```

G.8 Checking the checks

```
mutants: 15, caught 14 (3 seeds x 600 random items + directed phases each; with and without the assertions)
survivors are annotated in the table; see docs for the reason (B14 is an equivalent mutant)

B01  KILLED    PENABLE asserted already in the setup phase of a write                assertions:
    a_bresp_maps_slverr,a_first_cycle_is_setup,a_penable_drops without assertions (testbench only): NOT CAUGHT
B02  KILLED    write ignores APB wait states (leaves the access phase at once)       assertions:
    a_access_until_ready,a_bvalid_only_after_write,a_dir_stable without assertions (testbench only): testbench
B03  KILLED    read ignores APB wait states                                          assertions:
    a_access_until_ready,a_rvalid_only_after_read  without assertions (testbench only): testbench
B04  KILLED    APB write error swallowed: BRESP always OKAY                          assertions: a_bresp_maps_slverr
                              without assertions (testbench only): testbench
B05  KILLED    APB read error swallowed: RRESP always OKAY                           assertions: a_rresp_maps_slverr
                              without assertions (testbench only): testbench
B06  KILLED    write response dropped without waiting for BREADY                     assertions: a_bvalid_holds
                                    without assertions (testbench only): testbench
B07  KILLED    read data dropped without waiting for RREADY                          assertions: a_rvalid_holds
                                    without assertions (testbench only): testbench
B08  KILLED    read uses the write address                                           assertions: -
                                               without assertions (testbench only): testbench
B09  KILLED    write data is one beat stale                                          assertions: -
                                               without assertions (testbench only): testbench
B10  KILLED    WREADY asserted in IDLE as well (data accepted before an address)     assertions: a_aw_w_ready_together
                               without assertions (testbench only): testbench
B11  KILLED    PWRITE dropped in the access phase of a write                         assertions:
    a_bvalid_only_after_write,a_rresp_maps_slverr,a_setup_to_access_same without assertions (testbench only): testbench
B12  KILLED    write has priority over read when both are requested                  assertions: a_read_priority
                                  without assertions (testbench only): testbench
B13  KILLED    PSEL dropped in the setup phase of a read                             assertions: a_first_cycle_is_setup
                              without assertions (testbench only): NOT CAUGHT
B14  SURVIVED  read data captured without the PREADY qualifier (final value still cor assertions: -
                                               without assertions (testbench only): NOT CAUGHT
B15  KILLED    read data not registered: RDATA follows PRDATA combinationally is not  assertions: a_rdata_is_prdata
                                 without assertions (testbench only): testbench
```

Two of the bugs (PENABLE asserted in the setup phase, and PSEL dropped in the setup phase) are caught *only* by an assertion. The functional testbench can't see a setup-phase violation, because the slave model tolerates it. One (B14, read data captured without the PREADY qualifier) is an equivalent mutant: the value presented with RVALID at the end is unchanged.

G.9 Simulation commands and tools

```
make check      # lint, 8 seeds, coverage
make baseline   # as-found RTL vs the same testbench
make mutants    # injected bugs
```

You need Verilator 5.020, Python 3 and make, plus Yosys and OpenSTA for section H.

H. Synthesis, timing and performance results

Boundary: inputs 0.35 ns after the edge, outputs 0.15 ns before the next edge (assumption). T_min = 10 ns - WNS, verified at +/-2 %. `default` = `abc -D 10000`; `speed` = timing-driven ABC script (buffering + sizing). Power is not reported (no switching-activity file).

RTL	mapping	cells	area (um^2)	flip-flops	T_min (ns)	F_max (MHz)	method check
as-found (src/original)	default	592	5370	138	4.611	216.9	verified (WNS +0.092 / -0.092 ns)
as-found (src/original)	speed	782	6399	138	1.681	594.9	verified (WNS +0.034 / -0.033 ns)
fixed (src/axi2apb.v)	default	595	5378	138	4.611	216.9	verified (WNS +0.092 / -0.092 ns)
fixed (src/axi2apb.v)	speed	784	6406	138	1.681	594.9	verified (WNS +0.034 / -0.033 ns)

How to read the table: the timing-driven ABC script inserts fan-out buffers and sizes gates for delay, and Yosys' default `abc -D 10000` does neither, so the same RTL maps to a much slower netlist. The

comparison measures the tool flow, not a design change. The CH-001 fix changes the area by a few square micrometres and doesn't change T_min. Power is **not reported**, because there's no switching-activity file and OpenSTA's default activity isn't a measurement.

Item	Status
Cells, area, flip-flops, T_min (pre-layout, typical corner)	measured (table)
Frequency in silicon, power, area after place and route	NOT MEASURED
An earlier "x % faster, y % lower power than the unoptimised netlist" statement	not reproducible from the original repository (no baseline netlist) and replaced by the measured comparison above

---

I. Results, limitations and future improvements

I.1 Summary

With a two-line fix, the bridge passes a self-checking testbench with 23 bound assertions on 8 seeds, hits every cover point and every line, catches 14 of 15 injected bugs (the 15th is equivalent), and has a measured synthesis and timing result. The as-found RTL fails the same testbench on every error access.

I.2 Most meaningful outcomes

1. A real specification bug found and fixed (DECERR instead of SLVERR), with evidence from before and after.
2. Protocol assertions that catch bugs the functional testbench can't see (APB setup-phase violations).
3. A measured synthesis comparison in place of unmeasured performance claims.

I.3 Limitations

One outstanding transaction; no write strobes; APB3 only; pre-layout timing only; no power figure; and the AXI master and the APB slave are the testbench's own.

I.4 Future improvements

Support for WSTRB/PSTRB; outstanding transactions with a small response queue; a formal proof of the APB phase rules; one-hot state encoding with a measured area and timing trade-off; and gate-level simulation with a switching-activity file for a real power figure.

I.5 Interview talking points

- The APB setup and access phases, what PREADY does, and why PENABLE has to drop after the transfer.
- AXI response codes: why an APB error is SLVERR and not DECERR.
- Why AWREADY and WREADY are asserted together, and what that does to a master that sends W before AW.
- How an assertion catches a protocol bug that a data-checking testbench can't.
- Why the same RTL gives a 4.6 ns or a 1.7 ns period depending only on the ABC script, and what that says about reading synthesis numbers.

J. References and reproducibility

- AMBA AXI and ACE Protocol Specification (AXI4-Lite); AMBA APB Protocol Specification (APB3), Arm.
- Verilator, Yosys and OpenSTA documentation; SkyWater SKY130 `sky130_fd_sc_hd` liberty.
- Directory structure: `src/axi2apb.v` (fixed), `src/original/axi2apb_original.v` (as found), `verif/` (testbench, APB model, checker, binds), `scripts/` (build, campaign, README results), `ppa/`, `constraints/`, `results/`, `docs/portfolio/`, and `legacy/` (files from the original repository that aren't used).
- To reproduce: `make check`, `make baseline`, `make mutants`, `make ppa`. The source is at https://github.com/AbhijeetJ-512/axi2apb-bridge; my older public `AXI2APB` repository holds the as-found version.